



ChatProject (OOP)

Version 0.1

Jana Weschenfelder | Python Advanced | 29.04.2025

Projektarbeit 3

Dozentin: Frau Meyer

(Diese Dokumentation wird auch als README.md bereitgestellt.)

1. Einleitung

ChatProject ist ein GUI-basiertes Python-Programm mit dem mehrere WebSocket Clients über einen WebSocket Server Nachrichten austauschen können, das auf objektorientierter Programmierung (OOP) basiert und im Rahmen der „Python Advanced“ Weiterbildung entwickelt wurde.

Es handelt sich um ein Client-Server-Modell, bei dem mehrere Clients über einen zentralen Server Nachrichten austauschen können. Ziel war es, saubere OOP-Prinzipien und Clean Code einzuhalten und typische Fehler wie Redundanzen oder unsaubere Eingaben zu vermeiden.

2. Programmübersicht

2.1 Programmübersicht

Zunächst muss der WebSocket Server und danach der oder die WebSocket Clients gestartet werden.

2.2 Funktionsweise

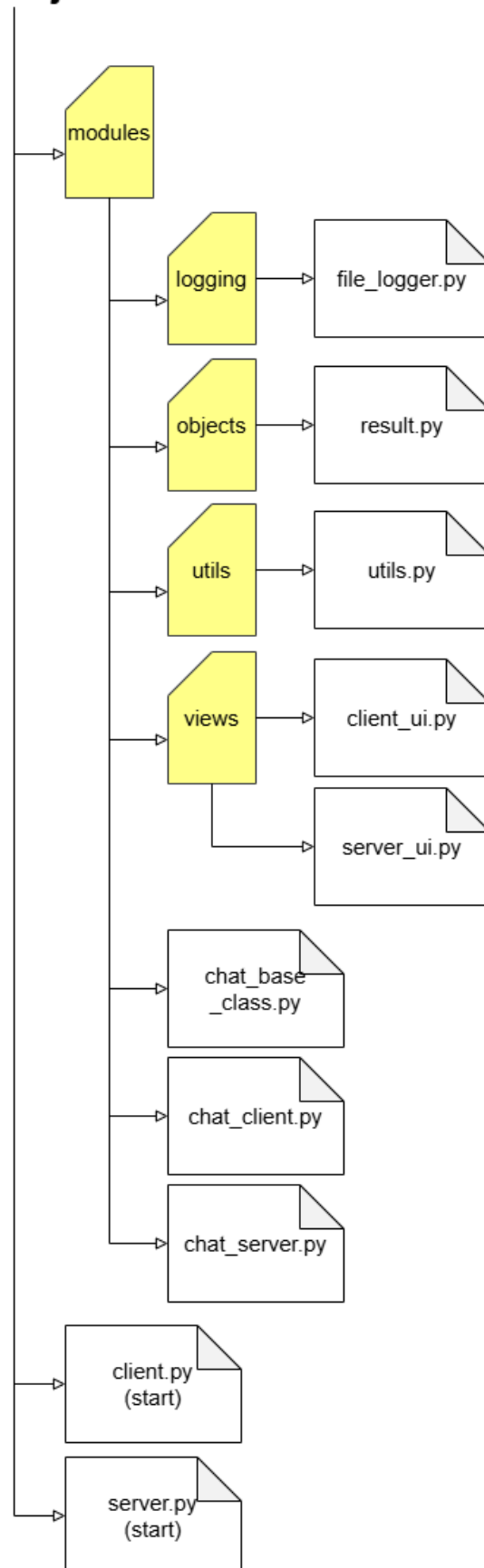
- Serverstart: Der Server wird gestartet und hört auf IP 127.0.0.1 und Port 5555.
- Clientstart: Clients verbinden sich zum Server (gleiche IP/Port) -> Handshake.
- Chatbetrieb:
 - Eingabe des Usernamens.
 - Eingabe und Versand von Nachrichten.
 - Nachrichten werden an alle verbundenen Clients verteilt.
 - Emoji-Shortcuts (z.B. ":") wird zu einem Lächeln) werden automatisch ersetzt.
- Beenden: Clients können den Chat sauber verlassen; Verbindungsabbrüche werden erkannt und verarbeitet.
- Verfügt über einen Logger, der in die Datei chat_project.log im Projektordner schreibt.

3. Verwendete Technologien

- Python 3.13
- Verwendete Module: sys, os, socket, select, typing, datetime, threading, tkinter

4. Programmstruktur

ChatProject



- /logging/file_logger.py: Logger, der in die Datei chat_project.log im Projektordner schreibt.
- /modules/objects/result.py: NamedTuple zum Speichern von Ergebnissen
- /modules/utils/utils.py: Hilfsfunktion zum Formatieren der WebSocket-Nachricht
- /modules/views/client_ui.py: Client GUI mit UI-Komponenten-Konfiguration
- /modules/views/server_ui.py: Server CLI UI -> zeigt einen Osterhasen beim Start
- /modules/chat_base_class.py: Basisklasse für WebSocket Kommunikation
- /modules/chat_client.py: Client-Logik für WebSocket Kommunikation
- /modules/chat_server.py: Server-Logik für WebSocket Kommunikation
- /client.py: Programmeinstiegspunkt zum Starten des Chat Clients
- /server.py: Programmeinstiegspunkt zum Starten des Chat Servers

5. Bedienung

1. Zuerst über ein Terminalfenster den Server starten mit: **python server.py**
2. Dann über ein anderes Terminalfenster den Client starten mit: **python client.py**
3. Im Client im Fenster oben einen Namen (Username) vergeben, dann auf Confirm klicken.
4. Anschließend unten bei Message eine Nachricht eingeben und Send drücken.
5. Für weitere Clients die Schritte 2. bis 4. wiederholen.
6. Zum Beenden den Button Beenden drücken.

6. Beispielausgabe

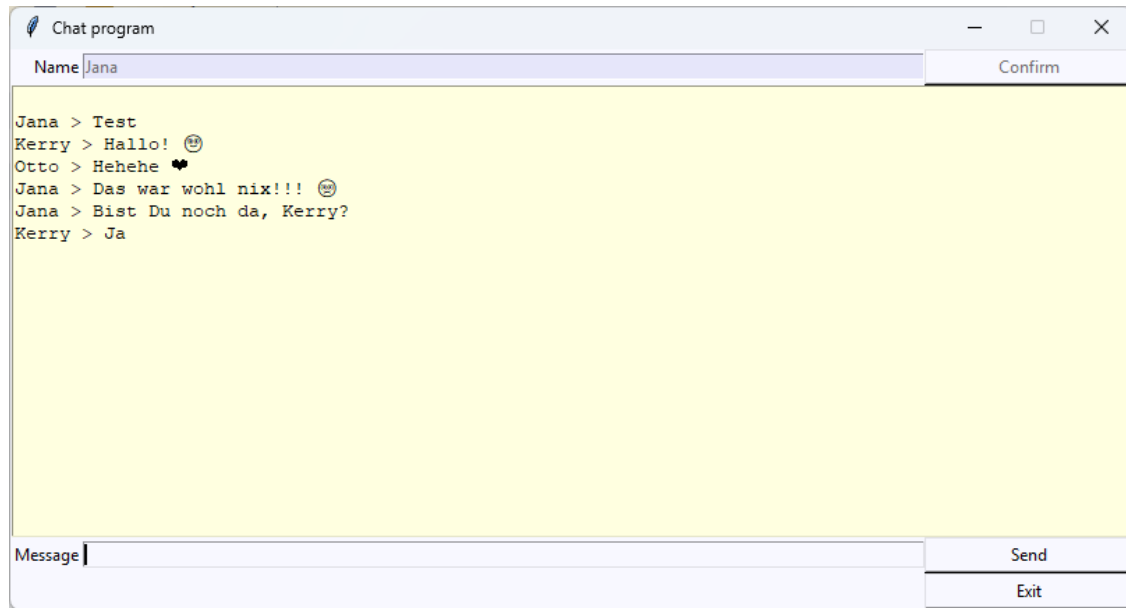
- Server:

```
(.env) PS C:\Users\wesch\PycharmProjects\ChatProject> python server.py

//
('>
/rr
*\))_

Listening on 127.0.0.1:5555
Established connection to 127.0.0.1:60423
Established connection to 127.0.0.1:60424
Established connection to 127.0.0.1:60427
Jana > Test
Kerry > Hallo! :-)
Otto > Hehehe <3
Jana > Das war wohl nix!!! :-(
127.0.0.1:60427 closed the connection
Jana > Bist Du noch da, Kerry?
Kerry > Ja
```

- Client:



7. Lizenz

Aktuell ist keine Lizenz vorgesehen. Es gilt kein Copyright, bitte aber das Programm pfleglich behandeln.

8. Besonderheiten/Aufgetretene Probleme

Es war angedacht, dass ein Client einem anderen Client auch eine Direktnachricht senden kann, zudem hätte man dort eindeutige IDs vergeben können. Der Versuch musste jedoch abgebrochen werden und auf die ursprüngliche, funktionierende Variante zurückgeschwenkt werden. Einen eigenen ClientPool zu implementieren, ist ein schwieriger Baustein und nicht so einfach wie ursprünglich gedacht, da er zwischen list (oder dict oder set), socket, usernames und IDs vermitteln muss.

9. Fazit/Ausblick

Man könnte noch einen richtigen ClientPool implementieren, der zusätzlich die Direktkommunikation via Usernamen und auch das Anzeigen des Client Status (welche User sind aktuell verfügbar?) anzeigt.

Zukünftige Erweiterung: Wetteranzeige durch externe REST-API bei Start und/oder auf Befehl.